

Mathematical Induction

Base Case: Show that $P(n_0)$ is true

↳ smallest input size
↳ first iteration of loop

Induction Hypothesis: Assume $P(k)$ is true for some k

Induction Step: Prove $P(k+1)$ is true using the assumption

Example,

Prove that sum of first n numbers of natural numbers is equal to $\frac{n(n+1)}{2}$

① Base Case:

$$n_0 = 1$$

$$\text{Sum} = 1$$

$$\frac{1(1+1)}{2} = 1 \quad (\text{TRUE})$$

② Induction Hypothesis:

Assume true for k

$$1+2+3+\dots+k = \frac{k(k+1)}{2}$$

③ Induction Step:

$$1+2+3+\dots+k+k+1 \stackrel{?}{=} \frac{k+1(k+2)}{2}$$

Use assumption from ②;

$$\frac{k(k+1)}{2} + k+1 \stackrel{?}{=} \frac{k+1(k+2)}{2}$$

$$\frac{1}{2}k^2 + \frac{1}{2}k + k + 1 \stackrel{?}{=} \frac{1}{2}(k^2 + 3k + 2)$$

$$\frac{1}{2}k^2 + \frac{3}{2}k + 1 = \frac{1}{2}k^2 + \frac{3}{2}k + 1$$

Hence,
Proven

Loop Invariant

- It is used to prove correctness of loops in algorithms
- It must hold (be true) at initialization, maintenance, and termination
- It is a property or set of properties of type boolean

Initialization:

- The Invariant holds prior to first iteration of the loop.

Maintenance:

- Assume the invariant holds before iteration k , then it must hold after the iteration $k+1$

Termination:

- The invariant holds when loop terminates

If invariant is true at initialization, maintenance & termination, then the algorithm is correct.

Example,

Loop Invariant:

max is maximum value
in $a[0 \dots i-1]$

↓
Property of
boolean type

```
def get_max(a):
    max = a[0]

    for i in range(1, len(a)):
        if a[i] > max:
            max = a[i]

    return max
```

Initialization:

Before $i = 1$,

$a[0 \dots i-1] \stackrel{?}{=} \text{max}$

$a[0 \dots 1-1] \stackrel{?}{=} \text{max}$

$a[0] = \text{max}$

(TRUE)

Maintenance:

At the start of iteration k ,

max is the maximum value in $a[0 \dots k-1]$
(our assumption)

In next iteration, the code updates max to $a[k]$ or doesn't. Regardless, max is maximum value in $a[0 \dots k]$

(TRUE)

Termination

$i = \text{len}(a)$

max is maximum value in $a[0 \dots \text{len}(a)-1]$

So, (TRUE)

Example,

Loop Invariant:

Elements in subarray
 $a[0 \dots i-1]$ are sorted

Initialization:

before $i = 1$ (first iteration)

$a[0 \dots 1-1] \stackrel{?}{=} \text{sorted}$

$a[0 \dots 0]$

$a[0]$

for ($i = 1$ to $n-1$)

{

key = arr[i];

$j = i-1$;

while ($j \geq 0$ and arr[j] > key)

{

arr[j+1] = arr[j];

$j = j-1$;

}

arr[j+1] = key;

}

A single element is trivially sorted

(TRUE)

Maintenance:

At start of iteration k ,
 $a[0 \dots k-1]$ is sorted. (Our assumption)

Then, $a[0 \dots k]$ is also sorted as while loop moves k^{th} element to correct position.

Termination

$i = \text{len}(a)$

Thus, $a[0 \dots \text{len}(a)-1]$ is also sorted when $i = \text{len}(a)$

Exercise (Introduction to Algorithms)

2.1.2

Loop Invariant:

$\text{sum} =$ summation of all elements in subarray
 $a[0 \dots i-1]$

Initialization:

before $i=0$, (first iteration)

$\text{sum} =$ summation of elements in $a[0 \dots 0-1]$

$=$ summation of elements in $a[0 \dots -1]$

That means $\text{sum} = 0$ as this subarray is empty. Thus,

(TRUE)

Maintenance:

At start of iteration k ,

Assume $\text{sum} =$ summation of elements in $a[0 \dots k-1]$

In next iteration, k^{th} element is also added thus sum becomes summation of elements in $a[0 \dots k]$

Termination

Loop terminates when $i = n$ & at that point sum is summation of elements in $a[0 \dots n-1]$